

Projet SAE S6 : Gestion de Bibliothèque - version FI

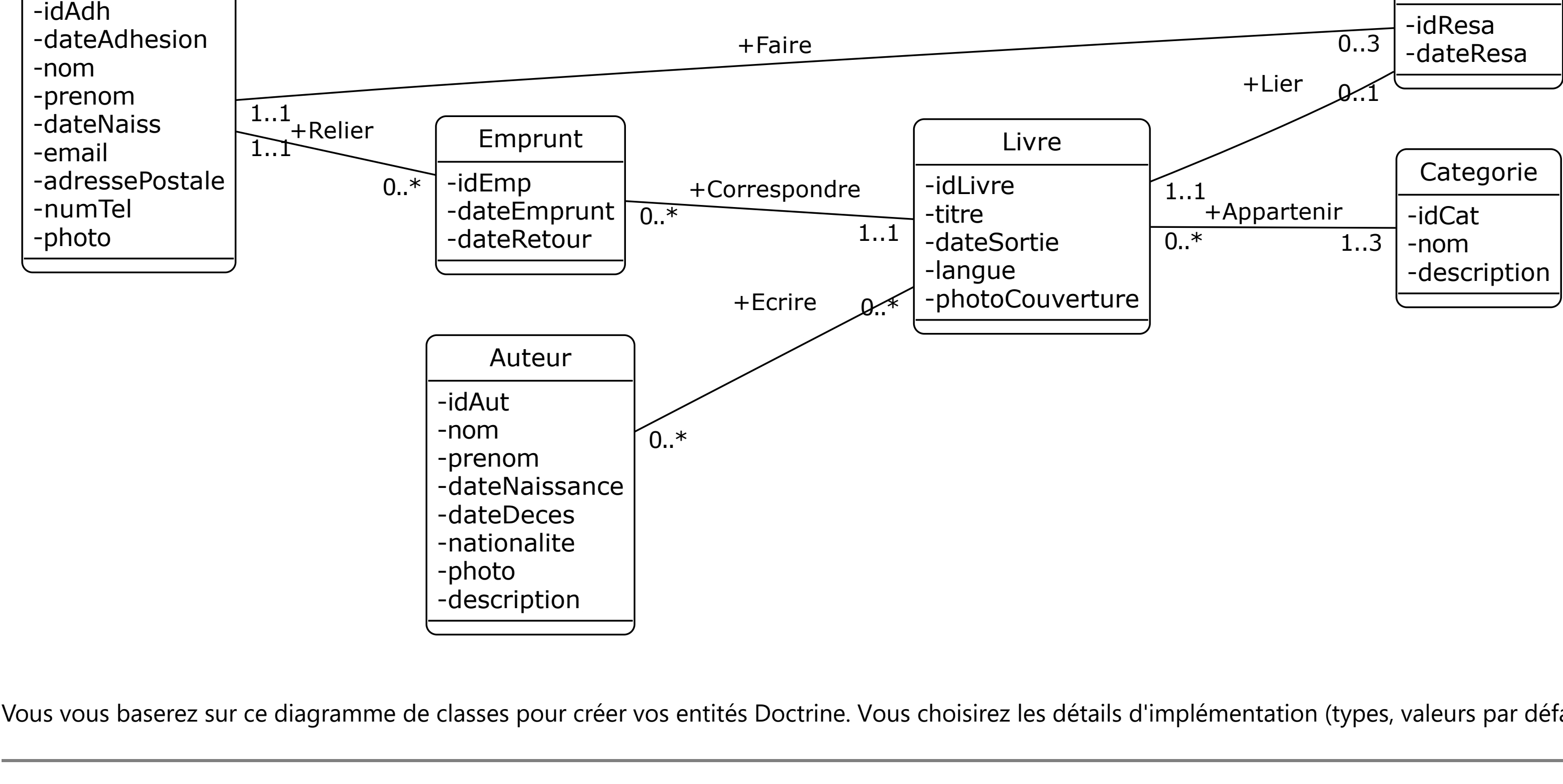
Présentation générale

Le but de ce projet est de réaliser une application web de **Gestion de Bibliothèque** en utilisant les technologies et méthodes mises en œuvre lors du POC (**outils et langages imposés**).

L'application se compose de trois briques :

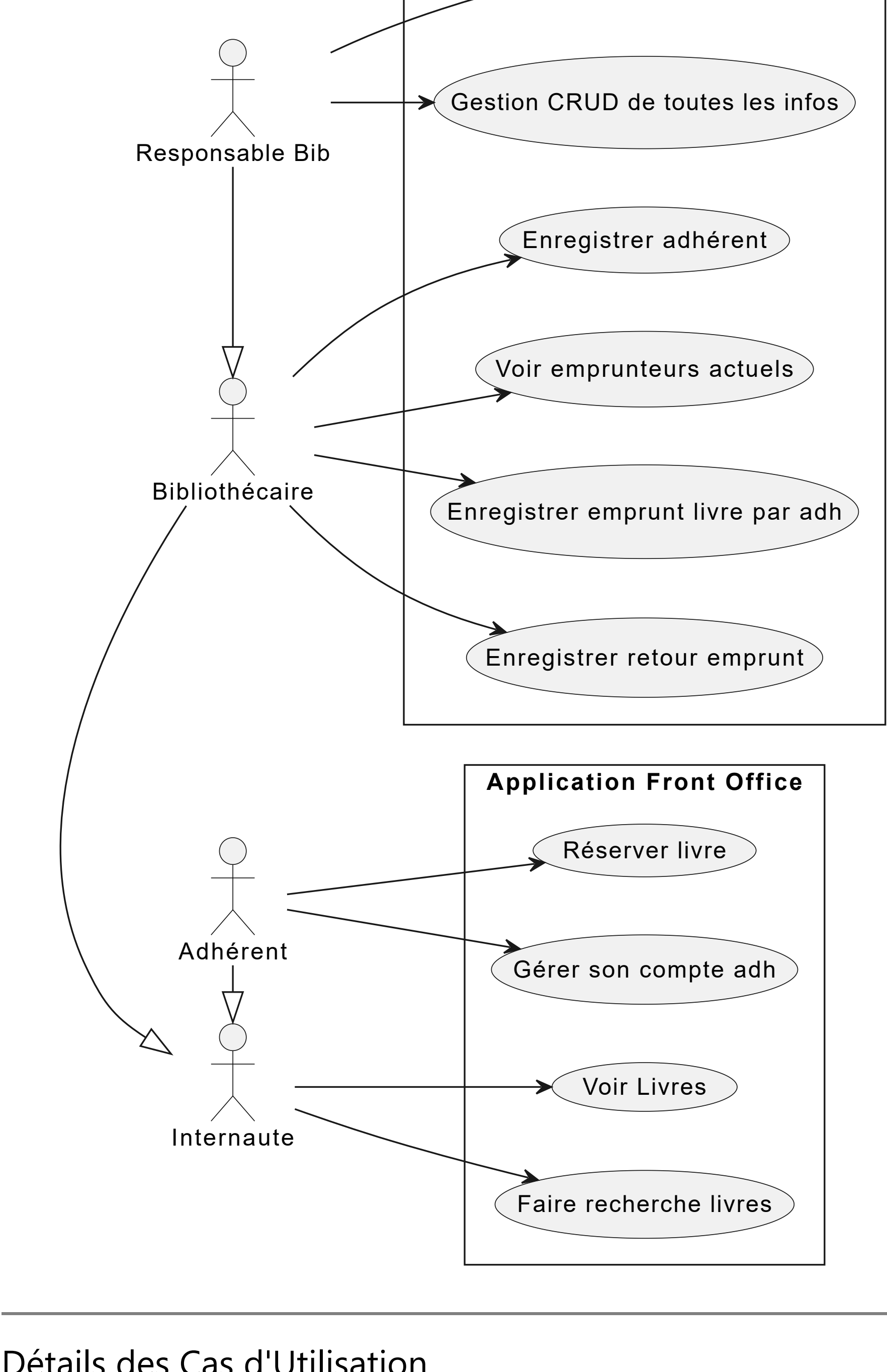
- **Back-office** (Symfony + EasyAdmin) : gestion des auteurs, livres, catégories, adhérents et emprunts.
- **API REST** (Symfony) : exposition des données et services pour le front-office.
- **Front-office** (Angular) : consultation du catalogue, recherche, réservation et gestion de compte adhérent.

Diagramme de Classes (minimal)



Vous vous bazerez sur ce diagramme de classes pour créer vos entités Doctrine. Vous choisirez les détails d'implémentation (types, valeurs par défaut, contraintes de validation, etc.) selon ce qui vous paraît le plus cohérent.

Diagramme (minimal) des Cas d'Utilisation



Détails des Cas d'Utilisation

Internaute (Front-office — accès public)

- Peut voir la liste des livres de la bibliothèque (avec pagination)
- Peut faire des recherches de livres (par titre, date de sortie, auteur, langue, catégorie, ...)
- Peut consulter la fiche détaillée d'un livre (auteur(s), catégorie(s), disponibilité)

Adhérent (Front-office — accès authentifié)

- Hérite de toutes les fonctionnalités de l'internaute
- Peut gérer son compte adhérent (consulter ses informations, modifier email, téléphone, adresse...)
- Peut voir l'historique de ses emprunts et ses réservations en cours
- Peut réserver un livre (pour un emprunt dans un futur proche)

Règles de gestion des réservations :

- 3 réservations maximum simultanées par adhérent
- L'adhérent peut librement annuler une réservation
- Une réservation qui se transforme en emprunt disparaît automatiquement
- Une réservation de plus de 7 jours expire automatiquement
- On ne peut pas réserver un livre déjà réservé par un autre adhérent
- On ne peut pas réserver un livre actuellement emprunté

Bibliothécaire (Back-office)

- Peut inscrire/créer un adhérent (et suspendre son inscription)
- Peut enregistrer les emprunts de livres par les adhérents
 - Un emprunt dure 15 jours maximum ; au-delà, il est considéré « en retard »
 - Un adhérent peut avoir **5 emprunts en cours** maximum
- Peut enregistrer les retours d'emprunts
- Peut voir la liste des emprunteurs actuels (à jour et en retard)
- Peut accéder au front-office comme un internaute

Responsable Bibliothèque (Back-office)

- Hérite de toutes les fonctionnalités du Bibliothécaire
- Dispose d'un CRUD complet sur toutes les entités (auteurs, livres, catégories, adhérents)
- Peut voir des statistiques sur les emprunts terminés (tableau de bord)
- Peut gérer les comptes bibliothécaires
- Peut accéder au front-office comme un internaute

Exigences techniques

Stack imposée

Composant	Technologie
Back-end	Symfony 7.x , PHP 8.4
Back-office	EasyAdmin 4.x
API	API REST manuelle (contrôleurs Symfony, groupes de sérialisation)
Front-end	Angular 21 (standalone components, signals, nouvelle syntaxe <code>@if/@for</code>)
Authentification	JWT (<code>lexik/jwt-authentication-bundle</code>)
BDD	MariaDB / MySQL
CSS	Framework CSS au choix (Bootstrap, Tailwind, ...)

Exigences Back-office (Symfony + EasyAdmin)

1. **Dashboard** : page d'accueil du back-office avec un tableau de bord affichant des statistiques (nombre de livres, emprunts en cours, emprunts en retard, adhérents actifs, etc.).
2. **CRUD configurés** : chaque entité doit avoir un CRUD controller EasyAdmin avec des champs correctement typés (`AssociationField`, `BooleanField`, `DateTimeField`, `ImageField`, etc.).
3. **Formulaire complexe** : au moins un formulaire géré « à la main » dans EasyAdmin — le formulaire de gestion d'emprunts permettant de sélectionner un adhérent et un ou plusieurs livres, tout en vérifiant la disponibilité des livres et la limite d'emprunt (5 max en cours).
4. **Gestion des mots de passe** : le champ mot de passe doit être correctement géré en édition (champ masqué, optionnel en modification, hachage uniquement si rempli).
5. **Rôles et accès** : le back-office doit appliquer une distinction entre Bibliothécaire (`ROLE_BIBLI`) et Responsable (`ROLE_ADMIN`). Le bibliothécaire ne doit pas voir les mêmes menus/CRUD que le responsable.
6. **Fixtures** : un jeu de données suffisant pour la démonstration (au moins 5 auteurs, 20 livres, 5 catégories, 10 adhérents, 15 emprunts variés).

Exigences API REST

1. **Endpoints publics** : liste des livres (avec pagination), détail d'un livre, liste des catégories, liste des auteurs.
2. **Endpoint de recherche avancée** : recherche de livres selon plusieurs critères combinables (titre, auteur, catégorie, langue, période de sortie).
3. **Endpoints protégés (JWT)** : profil de l'adhérent connecté, ses emprunts, ses réservations, création/annulation de réservation.
4. **Groupes de sérialisation** : utilisation de `#[Groups]` pour éviter les références circulaires et adapter la réponse au contexte (liste vs détail).
5. **Codes HTTP appropriés** : `200`, `201`, `400`, `401`, `403`, `404`, `409` (conflit de réservation), etc.
6. **CORS** : configuré correctement pour permettre les appels depuis le front Angular.

Exigences Front-office (Angular)

1. **Architecture** : composants standalone, injection via `inject()`, signaux (`signal`, `computed`), nouvelle syntaxe de templates (`@if`, `@for`, `@empty`).
2. **Pages publiques** :
 - Accueil avec présentation et statistiques générales
 - Catalogue des livres avec recherche multi-critères et pagination
 - Fiche détaillée d'un livre (auteurs, catégories, disponibilité)
 - Liste des auteurs / fiche auteur
3. **Pages authentifiées** (adhérent connecté) :
 - Tableau de bord personnel (emprunts en cours, réservations)
 - Gestion du profil (modification des informations personnelles)
 - Réservation d'un livre (avec feedback en cas d'erreur : déjà réservé, limite atteinte, etc.)
 - Annulation de réservation
4. **Authentification** :
 - Page de connexion / déconnexion
 - Intercepteur HTTP ajoutant le token JWT
 - Navigation conditionnelle selon l'état de connexion
 - Guard sur les routes protégées (redirection vers login si non authentifié)
5. **UX** : l'interface doit être soignée, responsive, et offrir un feedback utilisateur clair (messages de succès, d'erreur, spinners de chargement).

Exigences transversales

1. **Qualité du code** : code lisible, bien structuré, nommage cohérent (français ou anglais, mais pas un mélange des deux).
2. **Validation** : contraintes de validation côté serveur (Symfony Validator) sur les entités.
3. **Gestion des erreurs** : messages d'erreur explicites côté API et côté front.

Constitution des groupes

Les groupes de 3 étudiants sont constitués au choix des étudiants, en suivant les consignes des enseignants (qui peuvent réarranger les groupes au besoin).

Travail à effectuer

En suivant la logique du POC et avec les connaissances acquises en Symfony et Angular, développer l'application complète respectant le cahier des charges ci-dessus.

Une fois les fonctionnalités minimales en place, vous pouvez enrichir votre application de fonctionnalités complémentaires qui valoriseront votre travail lors de l'évaluation. Exemples :

- Système de notifications (emprunts bientôt en retard)
- Export PDF des fiches d'emprunt
- Filtres avancés avec sauvegarde des recherches
- Mode sombre / thème personnalisé
- Upload et gestion des photos (couvertures, auteurs)
- Historique des actions (logs)

Vous êtes libres d'utiliser des outils complémentaires (Git, Docker, ...) pour votre projet mais vous devrez être autonomes sur ces outils.

Livrables

L1 — Dépôt du code source

Le code source complet du projet doit être déposé sur Moodle **au plus tard le mercredi de la semaine 11 à 23h59**. Le dépôt contient :

- Le projet Symfony complet (`back_biblio/` ou nom de votre choix)
- Le projet Angular complet (`front_biblio/` ou nom de votre choix)
- Un fichier `README.md` à la racine avec les instructions d'installation et de lancement (prérequis, commandes, URL d'accès, **users/passwd** des acteurs présents dans le dump SQL)
- Un dump SQL de votre base de données contenant les fixtures de recette (`dump_recette.sql`)

L2 — Cahier de recette

Un document PDF structuré décrivant les scénarios de test que vous déroulerez lors de la soutenance. Format imposé :

#	Scénario	Prérequis	Actions	Résultat attendu	Rôle
1	Connexion adhérent	Adhérent existant en BDD	Saisir email + mdp, cliquer Connexion	Redirection vers dashboard, email affiché	Adhérent
2	...	...	...	...	...

Le cahier de recette doit couvrir **au minimum** :

- 3 scénarios front-office public (consultation, recherche)
- 3 scénarios front-office adhérent (connexion, réservation, annulation, profil)
- 3 scénarios back-office bibliothécaire (emprunt, retour, liste emprunteurs)
- 3 scénarios back-office responsable (CRUD, statistiques, gestion accès)
- 2 scénarios API REST (requêtes protégées et publiques; recup données du profil adhérent)

L3 — Documentation technique

Un document PDF (8-15 pages) contenant :

1. **Architecture du projet** : schéma global (front ↔ API ↔ BDD ↔ back-office), diagramme de classes final
2. **Choix techniques** : justification des choix d'implémentation (structure des entités, gestion des réservations, recherche, etc.)
3. **Guide d'installation** : étapes pour installer et lancer le projet sur une machine vierge
4. **Difficultés rencontrées** et solutions apportées
5. **Répartition du travail** dans le groupe (qui a fait quoi)

Calendrier des livrables

Livrable	Date limite	Support
L1 — Code source	semaine 11 : Mercredi 11/03, 23h59	Archive ZIP sur Moodle
L2 — Cahier de recette	semaine 11 : Jeudi 12/03, 14h00	PDF sur Moodle
L3 — Documentation technique	semaine 11 : Jeudi 12/03, 23h59	PDF sur Moodle
Soutenance	semaine 11 : Vendredi 13/03, dernier créneau SAE	Présentation live

Évaluation

La note finale est composée de trois parties :

Note collective — Soutenance de recette (coeff. 3)

Durée : **20 minutes de démonstration + 10 minutes de questions**.

Le groupe déroule son cahier de recette devant le client (enseignant). Le client évalue selon la grille suivante :

Critère	Points	Détail
Fonctionnalités Back-office	/5	CRUD, dashboard, formulaire complexe (emprunts), gestion des rôles
Fonctionnalités API REST	/3	Endpoints publics et protégés, recherche avancée, codes HTTP
Fonctionnalités Front-office	/5	Catalogue, recherche, réservation, profil, auth JWT
Qualité de l'UI/UX	/3	Design soigné, responsive, feedback utilisateur
Robustesse	/2	Gestion des erreurs, cas limites, validations
Fonctionnalités bonus	/2	Enrichissements au-delà du cahier des charges
Total	/20	

Méthode d'évaluation : Le client (enseignant) suit le cahier de recette du groupe et coche les scénarios réussis/échoués. Chaque scénario raté impacte la rubrique correspondante. Les 10 minutes de questions permettent de vérifier la compréhension technique de chaque membre.

Note collective — Documentation (coeff. 1)

Critère	Points	Détail
Cahier de recette	/8	Complétude, clarté, format respecté
Documentation technique	/12	Architecture, choix techniques, installation, répartition
Total	/20	

Note individuelle — QCM technique (coeff. 1)

Un QCM individuel sur papier portant sur les concepts techniques utilisés dans le projet :

- Symfony (entités, contrôleurs, sécurité, sérialisation)
- EasyAdmin (CRUD, champs, personnalisation)
- API REST (endpoints, JWT, codes HTTP)
- Angular (composants, services, signaux, intercepteurs, routing)

Ce QCM sera passé en **semaine 8** (vendredi 20 février 15h45 B219).

Récapitulatif

Évaluation	Type	Coeff.	Semaine
QCM technique	Individuelle	1	8
Soutenance de recette	Collective	3	11
Documentation	Collective	1	11

Conseils

- **Commencez par le modèle de données** : créez toutes les entités et les relations avant de construire le reste.
- **Travaillez en parallèle** : un membre peut avancer sur le back-office pendant qu'un autre travaille sur l'API et le troisième sur le front.
- **Testez au fur et à mesure** : ne laissez pas les tests à la fin. Chaque fonctionnalité terminée doit être vérifiée immédiatement.
- **Préparez votre recette tôt** : le cahier de recette vous sert aussi de checklist de progression.
- **Gérez les priorités** : implémentez d'abord les fonctionnalités de base (CRUD, liste, recherche) avant les cas complexes (emprunts, réservations, statistiques).
- **Préparez soigneusement votre recette** : chaque scénario doit être clair, précis, et réalisable. Répérez la recette à l'avance et assurez-vous que tous les membres du groupe maîtrisent les scénarios. Vérifiez également que l'ensemble de vos scénarios sont réalisables dans le temps imparti (avec une marge pour les imprévus).
- **Soignez votre présentation** : la soutenance est l'occasion de valoriser votre travail. Préparez une démonstration fluide, mettez en avant les points forts de votre projet, et soyez prêts à expliquer vos choix techniques.